

DUET: Endogenous Failure Detection and Robust Retry with Dual Action Spaces

Anonymous Authors

Abstract—Vision-language-action (VLA) policies map visual observations and language instructions to robot actions. With limited robot post-training data, they can drift from successful behavior during closed-loop execution. We study recoverable execution deviations, where retrying the demonstrated behavior remains feasible, rather than severe failures that require task-level replanning or a new manipulation strategy. Existing solutions often add external failure detectors and collect large amounts of human takeover data, making the resulting systems costly and loosely coupled. We introduce DUET (Dual-manifold Unified Execution and Transfer), a dual-action-head module that predicts both relative actions for smooth nominal control and absolute waypoints for trajectory re-entry. The endogenous disagreement between these action spaces detects when execution has become inconsistent with the demonstrated behavior, while the absolute waypoint-induced correction initiates a retry. Asymmetric training further makes the relative branch sensitive to deviations and the absolute branch robust to visual disturbances. We evaluate DUET in LIBERO simulation and real-world manipulation under recoverable visual, viewpoint, scene, and action disturbances. DUET preserves nominal success (TODO% vs. TODO% for the relative-only baseline) while achieving TODO% retry success, outperforming the strongest baseline by TODO percentage points.

I. INTRODUCTION

Vision-language-action (VLA) policies adapt pretrained vision-language models to map visual observations and language instructions directly to robot actions [1], [2], [3]. They are attractive because one policy can connect semantic instructions to continuous manipulation. Yet a policy trained with a small amount of robot data can leave the support of successful behavior during execution. We refer to this condition as an execution failure: the observation, robot-object configuration, or induced trajectory becomes out of distribution (OOD) with respect to the successful demonstrations. The cause may be an off-center grasp, a reachable object displacement, or a partial object slip, but it may also be external, such as an unexpected occlusion, viewpoint change, or displacement of the object or robot arm. In these cases, the policy may continue issuing locally plausible commands without recognizing that the current rollout should be retried. Detecting such deviations and returning to a demonstrated execution path can improve robustness without requiring a new task planner.

Mainstream approaches commonly introduce an external monitor, critic, verifier, or predictive model for failure detection [4]. When recovery is also required, they typically collect large amounts of human takeover data in which an

operator intervenes and demonstrates a corrective action [5], [6], [7]. This creates substantial data and deployment cost, especially when the disturbances span visual observations, object configurations, and robot poses. More importantly, the detector only indicates that execution has gone wrong, while the recovery controller is trained as a separate capability; their decisions need not be consistent. We ask whether the policy can use its own action predictions to recognize a recoverable OOD deviation and retry the learned behavior without additional failure labels or takeover demonstrations.

DUET is motivated by a simple asymmetry that we observed in preliminary experiments. Relative-action policies produce smooth and stable motion around the demonstrated trajectory, but once a grasp or pose deviates, a local command provides no explicit target for returning to the previous behavior. Absolute-action policies are less suited to fine local control, yet their predicted waypoints can point back toward a reachable portion of the demonstrated trajectory and make a retry possible. DUET therefore learns the same successful behavior in two action spaces: a relative head for nominal execution and an absolute head for trajectory re-entry. We use the disagreement between their predictions as an endogenous signal of OOD execution. The disagreement is expected to remain small when both heads recognize the same in-distribution behavior and to increase when the current rollout is no longer compatible with that behavior.

At inference time, DUET converts the absolute waypoint into a correction from the live pose and compares it with the relative action. A soft gate keeps relative control in the nominal regime and shifts toward the waypoint-induced correction when the disagreement indicates a recoverable deviation, allowing the robot to re-approach the demonstrated behavior and retry. The gate returns to relative control after the two predictions realign. This is a failure-aware retry mechanism, not a general recovery planner: DUET does not claim to solve irreversible failures, unreachable dropped objects, hardware faults, or cases that require discovering a new task strategy.

Our contributions are threefold:

- We formulate failure-aware retry as detecting recoverable OOD execution deviations and re-entering the support of successful behavior, separating this problem from severe failure recovery and task-level replanning.
- We propose DUET, a dual-action-head VLA that uses endogenous disagreement between relative and absolute action spaces to detect deviations and to activate an absolute waypoint-based retry while preserving smooth nominal control.

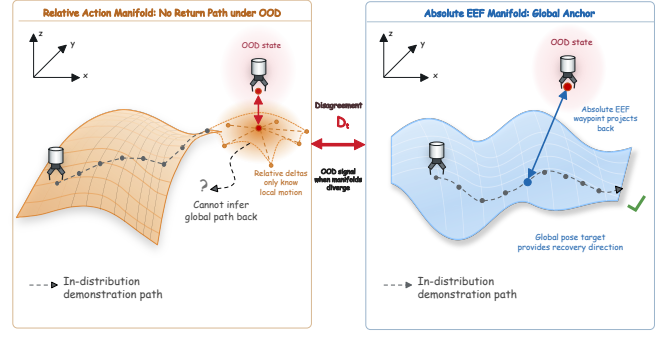
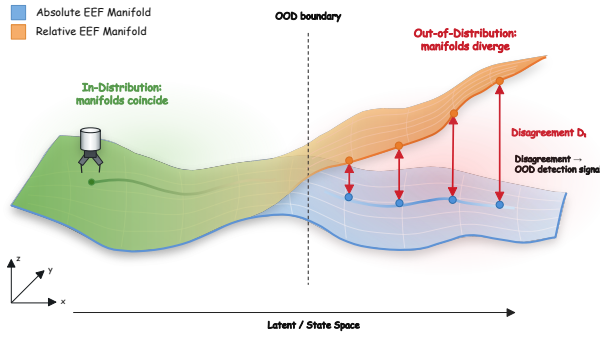


Fig. 1. Failure-aware retry with dual action spaces. Relative and absolute predictions agree during nominal execution but diverge when a recoverable deviation drives the rollout OOD, exposing an endogenous disagreement score D_t . Relative actions provide precise local motion, whereas absolute waypoints bring the robot back toward a demonstrated execution path for retry.

- We evaluate DUET in simulation and on a real robot across diverse visual, viewpoint, scene, and action disturbances, measuring both nominal robustness and retry success without additional human takeover data.

II. RELATED WORK

A. Failure Detection for Robot Policies

Runtime failure detection estimates whether a policy remains within the support of successful behavior. Diff-DAGger uses diffusion uncertainty to request expert intervention [8], while Sentinel monitors action inconsistency and task progress [9]. FAIL-Detect formulates failure detection as sequential OOD testing using only successful executions [4]; FIPER similarly combines representation novelty with action uncertainty without failure data [10].

VLA-specific methods use internal features or external foundation models. SAFE learns a detector from successful and failed rollouts [11], whereas AHA trains a VLM to detect and diagnose failures [12]. Such methods require additional supervision or a separate detector whose output is not a recovery action. DUET instead detects failures through disagreement between two action spaces learned within the policy, using OOD manifold inconsistency as both an alarm and a recovery signal.

B. Retry and Recovery from Execution Failures

Recovery methods commonly expand the training distribution with expert corrections. DAGger queries experts on policy-induced states [5]; intervention-based methods collect human takeovers [13], [6]; and IntervGen transforms limited interventions into corrective trajectories [7]. These approaches reduce compounding errors but still require additional recovery actions. DUET targets the narrower setting in which the original behavior remains feasible and a rollout can be retried after re-entering its demonstrated support.

Foundation-model systems instead use semantic supervisors. RACER learns from language-annotated recovery trajectories [14], while AHA and VLM controllers diagnose failures and guide downstream correction [12], [15]. Their detection and recovery components remain separately trained or prompted. DUET couples detection with a retry action

without extra recovery data: absolute states from successful demonstrations provide re-entry anchors, while cross-space disagreement determines when and how strongly to move toward them.

III. METHOD

A. Problem Formulation

At time t , the robot observes multi-view images o_t , a language instruction l , and its current proprioceptive state s_t . DUET is trained only on successful demonstrations and predicts the demonstrated behavior in two action representations over a horizon K :

$$\hat{\Delta}A_t = f_{\text{rel}}(o_t, l, s_t) \in \mathbb{R}^{K \times 7}, \quad (1)$$

$$\hat{W}_t = f_{\text{abs}}(o_t, l) \in \mathbb{R}^{K \times 7}. \quad (2)$$

Here, ΔA_t contains relative operational-space-control commands and W_t contains future absolute end-effector waypoints $[x, y, z, r_x, r_y, r_z, g]$. We decompose each command as $a_t = [a_{p,t}, a_{g,t}]$, where $a_{p,t} \in \mathbb{R}^6$ controls end-effector pose and $a_{g,t} \in \mathbb{R}$ controls the gripper. A successful trajectory naturally supplies both labels: its recorded delta actions supervise f_{rel} , while its future end-effector states supervise f_{abs} . The geometric analysis and fusion operate on a_p ; although the absolute head also predicts gripper state during training, the deployed a_g always comes from the relative head.

B. Dual Action Manifolds

For an instruction l , let $\mathcal{M}_l \subset SE(3)$ denote the end-effector state manifold traced by successful demonstrations, and let $T\mathcal{M}_l$ denote its tangent bundle. Under nominal execution, the relative head predicts a local motion vector

$$\hat{\delta}_{a_p,t} = f_{\text{rel}}(o_t, l, s_t)_p \in T_{p_t}\mathcal{M}_l, \quad (3)$$

where p_t is the current end-effector pose. This tangent representation is locally smooth and expressive, but it specifies only how to move from the current chart. Once a failure displaces the robot from the demonstrated support, the extrapolated vector field contains no explicit destination to which the robot should return.

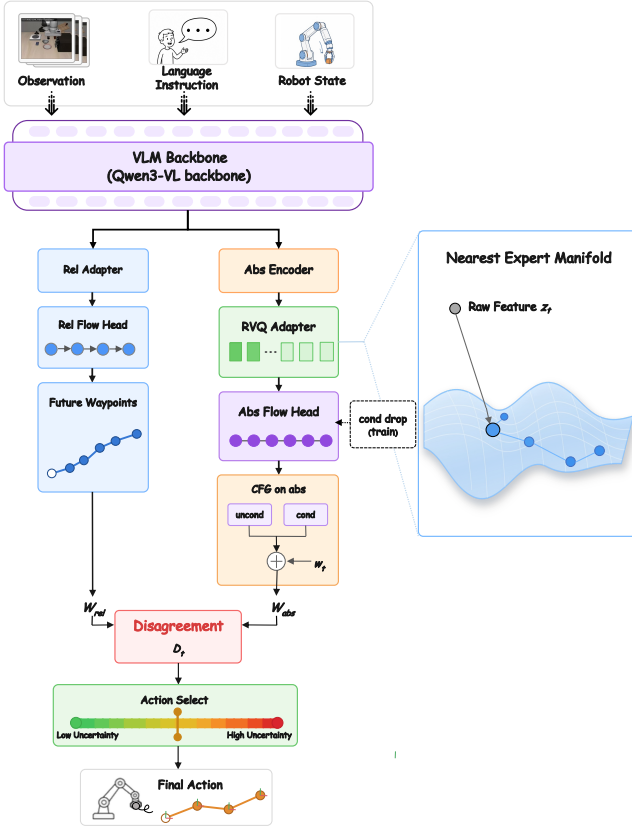


Fig. 2. DUET in four stages: a shared vision-language encoder, relative and absolute action heads, cross-space failure detection, and closed-loop recovery. The relative head provides state-aware local control; the explicitly state-free absolute head uses RVQ-anchored conditions to predict global waypoints. Their disagreement determines the transition between nominal execution and recovery.

The absolute head instead predicts a future coordinate

$$\hat{w}_{p,t} = f_{\text{abs}}(o_t, l)_p \in \mathcal{M}_l, \quad (4)$$

which acts as a visual-language-conditioned global anchor. To compare the two representations, we introduce a state-dependent transport map

$$c_t = \Psi_{p_t}(\hat{w}_{p,t}) \in T_{p_t}SE(3), \quad (5)$$

which converts the absolute waypoint into a local correction at the live pose. Along an in-domain trajectory, c_t and $\hat{\delta}_{a,p,t}$ describe the same demonstrated motion in two coordinate systems and should therefore align. When a failure drives execution OOD, the relative head extrapolates a local field while the absolute head remains anchored to a demonstrated destination, exposing a cross-space inconsistency.

This geometry couples detection and recovery without asserting a formal OOD guarantee: the norm of the inconsistency provides a scalar failure signal, whereas the transported vector c_t already provides the corresponding recovery direction. DUET can therefore change both when and where the controller moves using the same pair of predictions.

C. Dual-Head Architecture

Figure 2 shows the shared VLM and two flow-matching action heads [16], [17]. The relative head attends to the full hidden sequence and explicitly receives s_t through a numerical state encoder. It is therefore state-aware and serves as the primary controller during nominal execution. The absolute head receives no explicit current proprioception. When state tokens are appended to the causal VLM prompt, they are placed last and excluded from absolute-branch pooling, while the relative head can still attend to them. This separation prevents the absolute predictor from collapsing into an integration of the current state and preserves its role as a global reference.

The absolute branch pools its state-free hidden sequence H^0 and applies residual vector quantization (RVQ) [18], [19]:

$$Z_q = \text{RVQ}(\text{Pool}(H^0)). \quad (6)$$

RVQ maps continuous conditions to a discrete atlas learned from successful demonstrations, anchoring absolute predictions to recurrent visual-language modes rather than an unconstrained feature vector. Its residual norm is retained only as auxiliary telemetry and is not part of the primary failure score. The absolute flow head additionally uses classifier-free guidance (CFG) [20], enabling its conditional prediction to be softened toward the demonstrated waypoint marginal during recovery.

D. Disagreement-Preserving Learning

The two heads are supervised by the paired labels available in every successful trajectory. Both use flow-matching objectives, while RVQ contributes a commitment loss. The complete default objective is

$$\mathcal{L} = \lambda_{\text{rel}}\mathcal{L}_{\text{rel}} + \lambda_{\text{abs}}\mathcal{L}_{\text{abs}} + \lambda_{\text{vq}}\mathcal{L}_{\text{vq}}, \quad (7)$$

where the absolute labels are normalized future states and the relative labels are the recorded delta commands. We deliberately impose no cross-head consistency loss or distillation: forcing agreement everywhere would suppress the inconsistency needed for failure detection.

DUET instead specializes the branches through asymmetric visual augmentation. The absolute head and RVQ adapter train on both clean and perturbed observations, encouraging the global anchor to remain stable under degraded input. The relative head is optimized only on the clean subset $\mathcal{B}_{\text{clean}}$:

$$\mathcal{L}_{\text{rel}} = \frac{1}{|\mathcal{B}_{\text{clean}}|} \sum_{i \in \mathcal{B}_{\text{clean}}} \ell_{\text{FM}}(\hat{\Delta} a^{(i)}, \Delta a^{(i)}). \quad (8)$$

This gradient truncation preserves relative-head sensitivity while training the absolute branch as the robust reference, making cross-space disagreement more discriminative under visual OOD. For CFG training, the absolute head also replaces its quantized condition with learned null tokens on a subset of samples and learns both conditional and marginal waypoint flows.

E. Joint Failure Detection and Retry

At deployment, the policy returns both unnormalized action chunks. Let the live pose be $p_t = (x_t, R_t)$ and the pose component of the current absolute waypoint be $\hat{w}_{p,t} = (\hat{x}_t, \hat{R}_t)$. DUET implements the transport map in Eq. (5) as

$$c_t = \text{clip}\left(\left[\frac{(\hat{x}_t - x_t)/\sigma_p}{\text{Log}(\hat{R}_t R_t^{-1})/\sigma_R}\right], -1, 1\right), \quad (9)$$

where $\text{Log} : SO(3) \rightarrow \mathfrak{so}(3)$ gives the geodesic rotation correction and σ_p, σ_R convert pose error into action units. The cross-space disagreement is

$$D_t = \frac{\|c_t - \text{clip}(\hat{\delta}a_{p,t}, -1, 1)\|_2}{\sqrt{6}}. \quad (10)$$

This is the primary failure signal. We supplement it with a coarse state-distance term that measures normalized violation of the training-state bounds:

$$S_t = \text{mean}\left[\frac{\max(s_{\min} - p_t, 0)}{s_{\max} - s_{\min}} + \frac{\max(p_t - s_{\max}, 0)}{s_{\max} - s_{\min}}\right]. \quad (11)$$

The resulting failure score and recovery weight are

$$q_t = D_t + \lambda S_t, \quad (12)$$

$$\alpha_t = \sigma(k(q_t - \tau)). \quad (13)$$

DUET then interpolates between local execution and trajectory re-entry while retaining relative gripper control:

$$a_t = [a_{p,t}, a_{g,t}], \quad (14)$$

$$a_{p,t} = (1 - \alpha_t)\hat{\delta}a_{p,t} + \alpha_t c_t, \quad (15)$$

$$a_{g,t} = \hat{\delta}a_{g,t}, \quad (16)$$

where $[\hat{\delta}a_{p,t}, \hat{\delta}a_{g,t}]$ is the relative-head command. Thus relative control dominates along the demonstrated support, while a rising failure score shifts a_p toward the absolute re-entry direction; control returns smoothly after the two representations realign. When requesting a new chunk, the previous fusion weight also softens the absolute-head CFG scale,

$$w_t = w_{\max} - (w_{\max} - w_{\min})\alpha_{t-1}. \quad (17)$$

The disagreement D_t therefore determines *when* a retry should begin, while the vector c_t determines *how* to re-enter the demonstrated behavior.

IV. EXPERIMENTS

A. Benchmark and Data

We evaluate on LIBERO [21], using the StarVLA LeRobot data pipeline. The dual-label data configuration uses current plus future state indices $0, \dots, 8$, while action labels use the native delta OSC actions. The default draft configuration trains on the `libero_10_hybrid` mixture, with planned extensions to `libero_all_hybrid`. For few-shot experiments, the data loader limits demonstrations per task through `max_demos_per_task`.

TODO: Insert gate response plot.

Expected curves: injected perturbation interval, disagreement D_t , state distance S_t , fusion weight α_t , and CFG scale w_t over time.

Fig. 3. Planned closed-loop telemetry visualization for perturbation recovery experiments.

TABLE I

LIBERO NOMINAL AND FAILURE-AWARE RETRY SUCCESS RATES. ALL QUANTITATIVE ENTRIES ARE TODO PLACEHOLDERS AND MUST BE FILLED AFTER EVALUATION.

Method	Demos/task	Clean SR	Failure Recovery SR
Relative-only VLA	TODO	TODO	TODO
Absolute-only waypoint	TODO	TODO	TODO
Fixed fusion, $\alpha = 0.5$	TODO	TODO	TODO
DUET adaptive	TODO	TODO	TODO

B. Implementation Details

DUET is implemented as `QwenGR00THybrid` in `StarVLA`. The backbone is `Qwen-VL` with hidden dimension 2048. The relative branch uses a 16-layer DiT-B flow-matching head. The absolute branch uses an 8-layer DiT-B flow-matching head with 16 RVQ condition tokens of output dimension 1024. RVQ uses three residual quantizers with codebook size 512 and code dimension 256. We use visual augmentation probability $p_{\text{aug}} = 0.3$, CFG condition-dropout probability $p_{\text{drop}} = 0.15$, and disable the optional learned gate by default. Training uses AdamW and the schedule specified in the repository configuration.

C. Failure Detection and Recovery Protocol

We evaluate recoverable policy-induced and externally induced failures. The former include off-center grasps, partial slips, and reachable object displacements; the latter include camera occlusion, viewpoint changes, and moderate displacement of the object or robot arm. For controlled simulation, we additionally inject action-space perturbations with fixed start time, duration, and magnitude. We record detection accuracy and latency, retry success and time, action traces, dual-branch disagreement, state-distance score, fusion weight, and CFG scale.

D. Main Results

Table I will report nominal and failure-aware retry success rates. Relative-only control should remain competitive during nominal execution but be less robust after recoverable deviations, while DUET should improve retry success by increasing the trajectory re-entry weight only when needed.

E. Ablations

The ablations in Table II isolate condition anchoring, guidance adaptation, branch specialization, state prompt injection, adaptive fusion, and an optional learned gate trained with synthetic state perturbations. Additional metrics include failure-detection accuracy and latency, mean retry time, maximum deviation after the injected disturbance, and area under the α_t curve.

TABLE II

PLANNED ABLATIONS. TODO ENTRIES SHOULD BE FILLED WITH SUCCESS RATE, RECOVERY RATE, AND TELEMETRY STATISTICS.

Ablation	Clean SR	Failure Recovery SR
DUET full	TODO	TODO
No RVQ anchoring	TODO	TODO
No CFG adaptation	TODO	TODO
No relative gradient truncation	TODO	TODO
State-to-VLM disabled	TODO	TODO
DUET + learned gate	TODO	TODO
Fixed $\alpha = 0$	TODO	TODO
Fixed $\alpha = 1$	TODO	TODO

V. LIMITATIONS

DUET assumes access to absolute state labels aligned with the relative action chunks. This is natural in LIBERO and many robot logs, but may require additional calibration or state estimation on real hardware. The closed-loop gate currently uses simple min-max state statistics as a coarse proxy for distance to the demonstrated state manifold, which may be insufficient for highly multi-modal workspaces. The absolute branch is deliberately stateless; while this supports trajectory re-entry, it may reduce waypoint accuracy in tasks where the current robot state contains task-relevant information not recoverable from images. Most importantly, DUET is designed for recoverable deviations and retries, not irreversible failures, hardware faults, or task-level replanning. Finally, all quantitative claims remain pending until the planned simulation and real-world disturbance experiments are completed.

VI. CONCLUSION

We presented DUET, a dual-branch VLA policy for endogenous failure detection and robust retry. By learning relative actions for smooth nominal control and absolute waypoints for trajectory re-entry, DUET uses cross-space disagreement to detect recoverable OOD deviations and to activate a retry direction without an external detector or additional human takeover demonstrations. Its scope is deliberately limited: when the task remains feasible, DUET can bring execution back toward demonstrated behavior; it is not a general planner for irreversible failures.

ACKNOWLEDGMENT

TODO: Add acknowledgments, funding, compute resources, and any required dataset or codebase acknowledgments after the author list is finalized.

REFERENCES

- [1] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choremanski, T. Ding, D. Driess, A. Dubey, C. Finn, P. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, K.-H. Lee, S. Levine, Y. Lu, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich, "RT-2: Vision-language-action models transfer web knowledge to robotic control," *arXiv preprint arXiv:2307.15818*, 2023.
- [2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, T. Lampe, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, C. Finn, and P. Liang, "OpenVLA: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024.
- [3] StarVLA Community, "StarVLA: A lego-like codebase for vision-language-action model developing," <https://github.com/starVLA/starVLA>, 2026.
- [4] C. Xu, T. K. Nguyen, E. Dixon, C. Rodriguez, P. Miller, R. Lee, P. Shah, R. Ambrus, H. Nishimura, and M. Itkina, "Can we detect failures without failure data? uncertainty-aware runtime failure detection for imitation learning policies," in *Proceedings of Robotics: Science and Systems*, 2025.
- [5] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, 2011.
- [6] H. Liu, S. Nasiriany, L. Zhang, Z. Bao, and Y. Zhu, "Robot learning on the job: Human-in-the-loop autonomy and learning during deployment," in *Proceedings of Robotics: Science and Systems*, 2023.
- [7] R. Hoque, A. Mandlekar, C. R. Garrett, K. Goldberg, and D. Fox, "IntervGen: Interventional data generation for robust and data-efficient robot imitation learning," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2024, pp. 2840–2846.
- [8] S.-W. Lee, X. Kang, and Y.-L. Kuo, "Diff-DAGger: Uncertainty estimation with diffusion policy for robotic manipulation," in *IEEE International Conference on Robotics and Automation*, 2025.
- [9] C. Agia, R. Sinha, J. Yang, Z. Cao, R. Antonova, M. Pavone, and J. Bohg, "Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress," in *Proceedings of the Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270, 2025, pp. 689–723.
- [10] R. Römer, A. Kobras, L. Worbis, and A. P. Schoellig, "Failure prediction at runtime for generative robot policies," in *Advances in Neural Information Processing Systems*, 2025.
- [11] Q. Gu, Y. Ju, S. Sun, I. Gilitshchenski, H. Nishimura, M. Itkina, and F. Shkurti, "SAFE: Multitask failure detection for vision-language-action models," in *Advances in Neural Information Processing Systems*, vol. 38, 2025.
- [12] J. Duan, W. Pumacay, N. Kumar, Y. R. Wang, S. Tian, W. Yuan, R. Krishna, D. Fox, A. Mandlekar, and Y. Guo, "AHA: A vision-language-model for detecting and reasoning over failures in robotic manipulation," in *International Conference on Learning Representations*, 2025.
- [13] J. Spencer, S. Choudhury, M. Barnes, M. Schmittle, M. Chiang, P. Ramadge, and S. Srinivasa, "Learning from interventions: Human-robot interaction as both explicit and implicit feedback," in *Proceedings of Robotics: Science and Systems*, 2020.
- [14] Y. Dai, J. Lee, N. Fazeli, and J. Chai, "RACER: Rich language-guided failure recovery policies for imitation learning," in *IEEE International Conference on Robotics and Automation*, 2025, pp. 15 657–15 664.
- [15] H. Chen, Y. Yao, R. Liu, C. Liu, and J. Ichnowski, "Automating robot failure recovery using vision-language models with optimized prompts," *arXiv preprint arXiv:2409.03966*, 2024.
- [16] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, 2024.
- [17] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, J. Jakubczak, T. Jones, A. Khazatsky, S. Levine, A. Li-Bell, Y. Lu, S. Nair, K. Pertsch, Q. V. Xu, F. Xia, and T. Xiao, " π_0 : A vision-language-action flow model for general robot control," *arXiv preprint arXiv:2410.24164*, 2024.
- [18] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, "Neural discrete representation learning," in *Advances in Neural Information Processing Systems*, 2017.
- [19] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund, and M. Tagliasacchi, "Soundstream: An end-to-end neural audio codec," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2022.
- [20] J. Ho and T. Salimans, "Classifier-free diffusion guidance," *arXiv preprint arXiv:2207.12598*, 2022.
- [21] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," in *Advances in Neural Information Processing Systems*, 2023.